

Big Data - Final Project Report

Predicting storm damage severity from NOAA weather events
(1970–2025)

Team 29

Lana Ermolaeva (Tester and Technical Writer)

Stepan Vagin (ML Specialist)

Ramil Shakirzyanov (Data Scientist)

Ivan Ershov (Data Engineer)

May 6, 2026

Contents

1	Introduction	3
1.1	Business Objectives	3
2	Data Description	3
2.1	Data Characteristics	3
3	Architecture of Data Pipeline	4
4	Data Preparation	4
4.1	ER Diagram	5
4.2	A sample from the database	5
4.3	Creating Hive tables and preparing the data for analysis	5
5	Data Analysis	5
5.1	Insight 1: Financial Impact by Storm Type	6
5.2	Insight 2: Validation of Custom Severity Metric	6
5.3	Insight 3: Geospatial Risk (Extreme Events by State)	7
5.4	Insight 4: Time Series Trend of Total Damage	7
5.5	Insight 5: Human Cost vs. Economic Cost	8
5.6	Insight 6: Seasonality of Storms	9
6	ML Modeling	9
6.1	Feature extraction and data preprocessing	9
6.2	Training and fine-tuning	10
6.3	Evaluation	11
6.4	Result persistence and downstream availability	12
6.5	PDA conclusions	12
7	Data Presentation	12
7.1	Tab 1: Collection & Ingestion (Operational View)	12
7.2	Tab 2: Exploratory Data Analysis (EDA)	13
7.3	Tab 3: Predictive Data Analysis (PDA)	14
8	Conclusion	15
8.1	Summary of the report	15
9	Reflections on own work	16
9.1	Challenges and difficulties	16
9.2	Recommendations	17
9.3	Table of contributions	17

1 Introduction

1.1 Business Objectives

Extreme weather events are among the most destructive forces on the planet, causing an estimated \$300 billion in economic losses globally each year and intensifying as climate change accelerates. Governments, insurers, and humanitarian organizations worldwide face a critical need to shift from reactive disaster response toward predictive, data-driven preparedness, yet the sheer volume and complexity of weather records makes this a fundamentally big data challenge. The United States NOAA Storm Events Database, covering over 650,000 severe weather events across 24 years, offers one of the richest longitudinal records of storm impact on Earth and serves as a powerful proxy for this global problem. Our objective is to build an end-to-end distributed big data pipeline that analyzes historical storm patterns and trains machine learning models to predict storm damage severity, enabling emergency agencies to pre-position resources more efficiently, helping insurers assess risk exposure earlier, and equipping policymakers with the evidence needed to prioritize infrastructure investment in the most vulnerable regions.

2 Data Description

2.1 Data Characteristics

The dataset is built from the NOAA Storm Events Database. [Link to the dataset](#)

- **Total Records:** 2,012,795
- **Size:** 710.41 MB
- **Features:** event_id, state, event_type, damage_property, damage_crops, deaths_direct, deaths_indirect, injuries_direct, injuries_indirect, begin_date, end_date.
- **Target Column:**

$S = \text{damage_property} + \text{damage_crops} + a \cdot (\text{deaths_direct} + \text{deaths_indirect}) + b \cdot (\text{injuries_direct} + \text{injuries_indirect})$, $a=1000000$, $b=50000$

$$\text{severity} = \begin{cases} 0 & \text{if } S = 0 \\ 1 & \text{if } 0 < S \leq 25\text{th percentile} \\ 2 & \text{if } 25\text{th} < S \leq 75\text{th percentile} \\ 3 & \text{if } 75\text{th} < S \leq 95\text{th percentile} \\ 4 & \text{if } S > 95\text{th percentile} \end{cases}$$

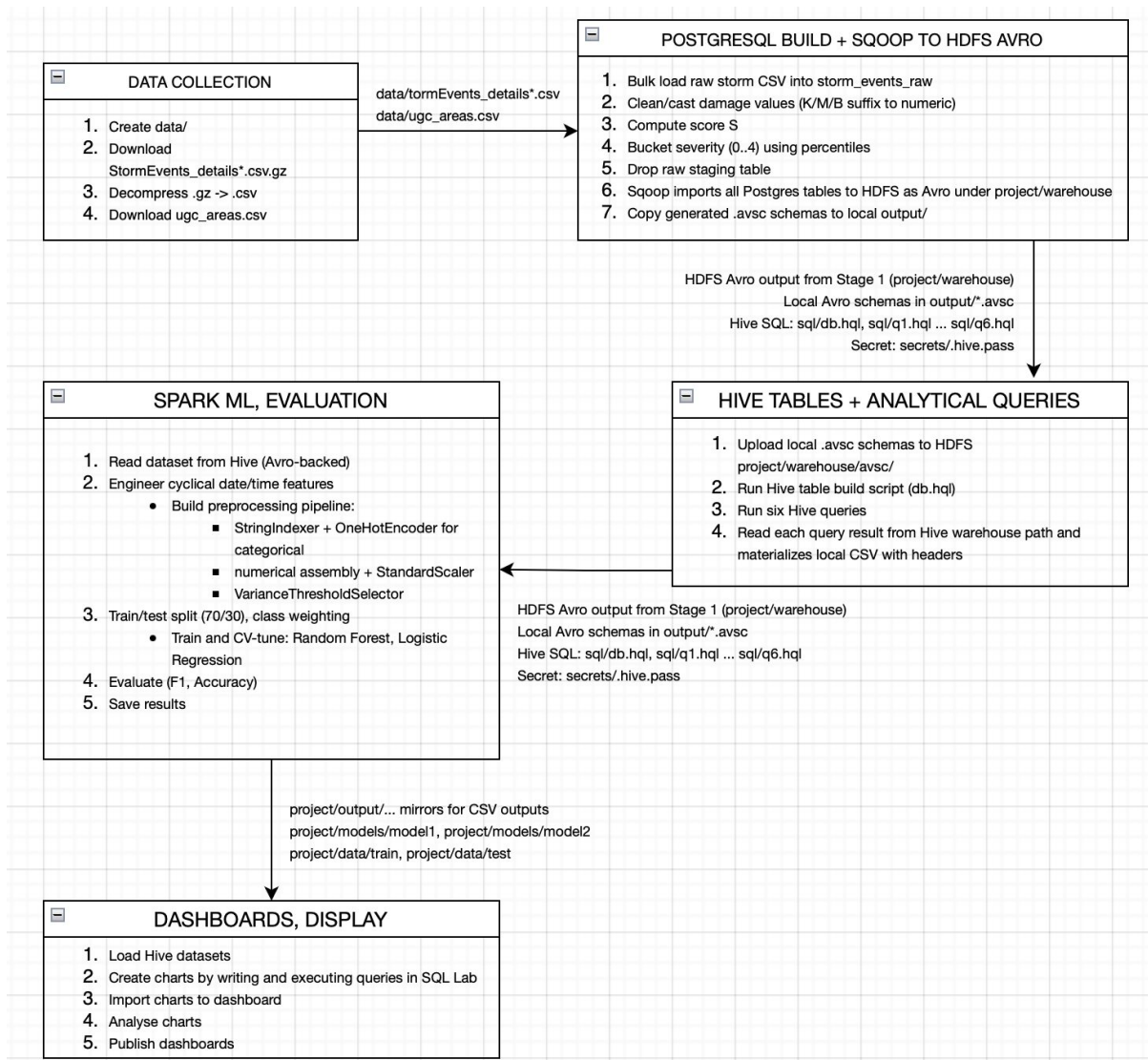
To satisfy stakeholders needs and be a ML-solvable task our "severity" needed to be:

1. One single number, perfectly 0-4
2. Covering material and human loss (direct and indirect, deaths and injuries)
3. Financially interpretable
4. Handling zero-impact events

5. Balancing class distribution

We constructed it merging best practices. Conway Disaster Impact Scale as a base (satisfies 1 and partially 2), included injuries like in UDSCS (for complete 2), used VSL with 1:20 (death:injuries) to get money equivalent to serve 3, 1 and 2). We overcame UDSCS fixed log-scale limitation on NOAA skewed data by using percentile cutpoints (for 4 and 5).

3 Architecture of Data Pipeline

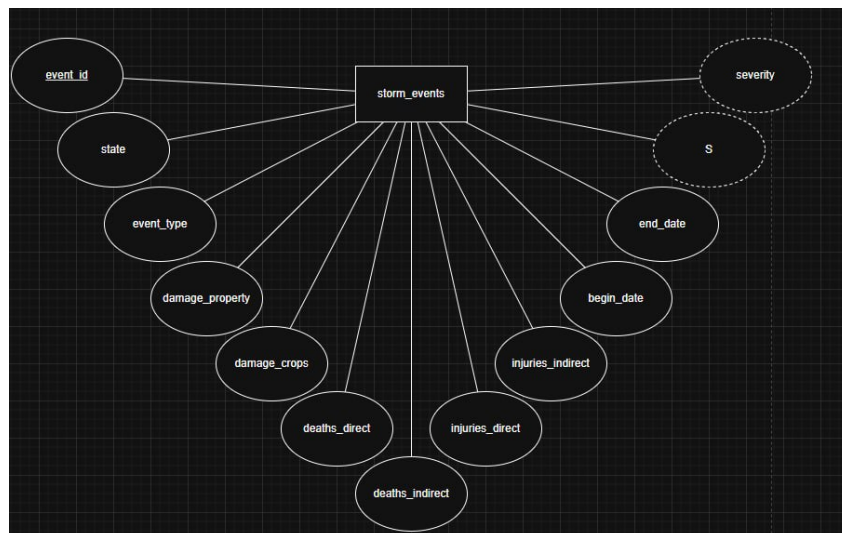


4 Data Preparation

In raw rows were loaded into `storm_events_raw`, then cleaned and transformed into `storm_events`: parsed money fields (`damage_property`, `damage_crops`) from K/M/B suffixes into numeric values, replaced empty injury and death counts with 0, parsed timestamps from text columns, filtered out invalid damage formats and empty state.

A custom severity score S was computed. A categorical severity label (0–4) was derived using quantiles (p_{25} , p_{75} , p_{95}) of S . Temporary raw table was dropped after transformation.

4.1 ER Diagram



4.2 A sample from the database

event_id	event_type	damage_property	damage_crops	deaths_direct	deaths_indirect	injuries_direct	injuries_indirect	begin_date	end_date	s	severity	state
1291745	Thunderstorm Wind	40000	0	0	0	0	0	2025-09-11	2025-09-11	40000	2	ALABAMA
1291744	Thunderstorm Wind	1000	0	0	0	0	0	2025-09-11	2025-09-11	1000	1	ALABAMA
1295519	Flood	0	0	0	0	0	0	2025-09-24	2025-09-24	0	0	ALABAMA
1295517	Flash Flood	0	0	0	0	0	0	2025-09-19	2025-09-19	0	0	ALABAMA

Figure 1: Sample

4.3 Creating Hive tables and preparing the data for analysis

To ensure performant Online Analytical Processing (OLAP), the raw AVRO files ingested via Sqoop were transformed in Apache Hive. An external table `storm_events_part_buck` was created. To optimize Big Data queries, the data was partitioned by state (reducing disk I/O via partition pruning) and bucketed by event_id into 8 buckets (optimizing future shuffle/join operations in Spark).

5 Data Analysis

The Exploratory Data Analysis (EDA) was performed using HiveQL on the partitioned and bucketed table to extract 6 key business insights prior to Machine Learning preprocessing. The results were aggregated in Hive and exported to Apache Superset to build the visual dashboard.

5.1 Insight 1: Financial Impact by Storm Type

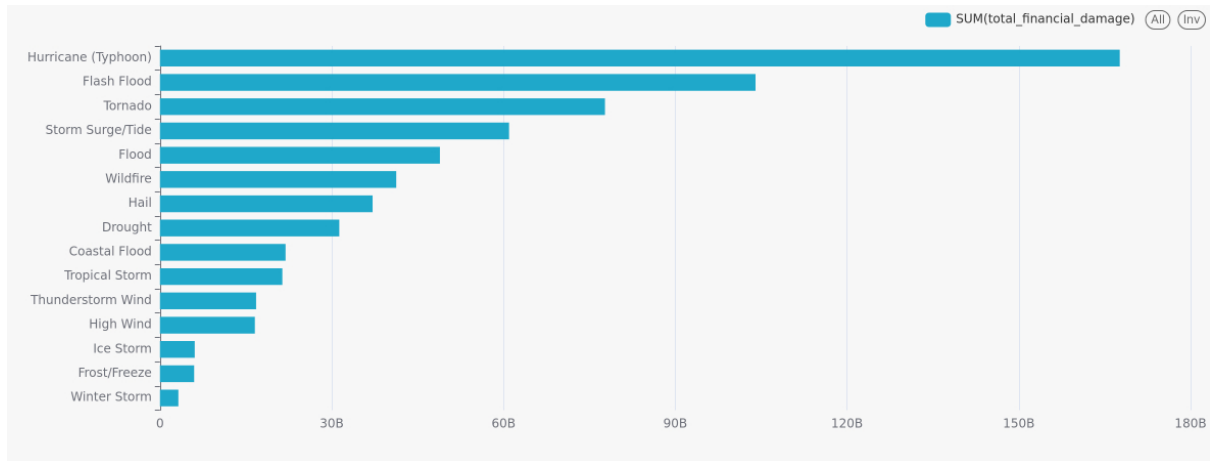


Figure 2: Total Damage by Event Type

Interpretation: This insight identifies the "economic killers." The horizontal layout reveals that **Hurricane (Typhoon)** is the most financially destructive event, accounting for approximately \$168 Billion in total damage. Water-related displacement events like **Flash Flood** (\$104B) and **Storm Surge/Tide** (\$78B) dominate the top five alongside **Tornado** events. This confirms to insurance stakeholders that hydro-meteorological risks carry significantly higher financial exposure than standard wind or winter weather.

5.2 Insight 2: Validation of Custom Severity Metric

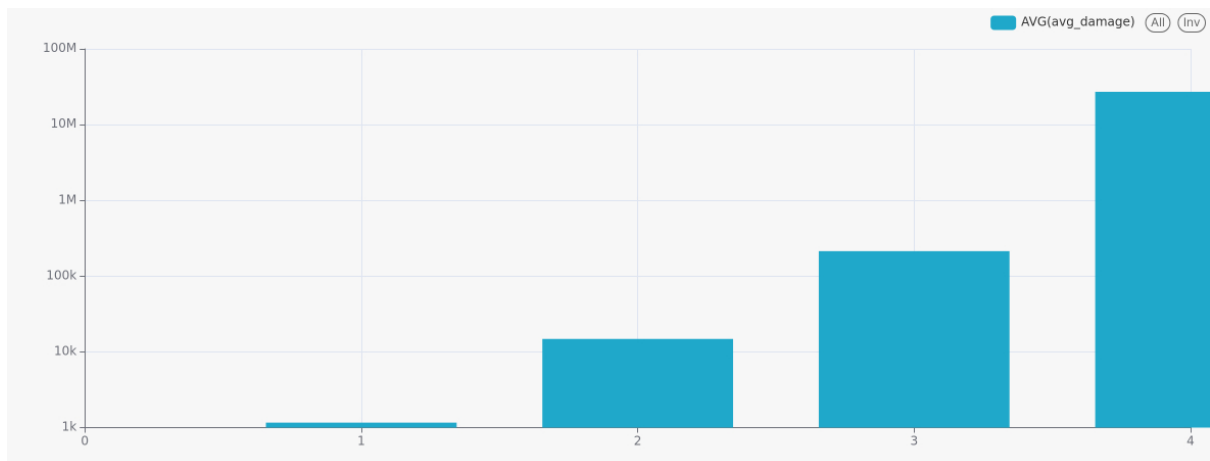


Figure 3: Average Damage per Severity Level (Log Scale)

Interpretation: This chart mathematically validates the target variable labeling logic using a logarithmic Y-axis. After filtering for non-zero events, we observe a perfect stratified progression across all five classes. Average damage scales exponentially from **Severity 1** (\$1.1k) to **Severity 4** (\$27 Million). This proves our percentile-based "tail

events” classification successfully separates minor incidents from catastrophic shocks, providing a clean ordinal scale for the machine learning model.

5.3 Insight 3: Geospatial Risk (Extreme Events by State)

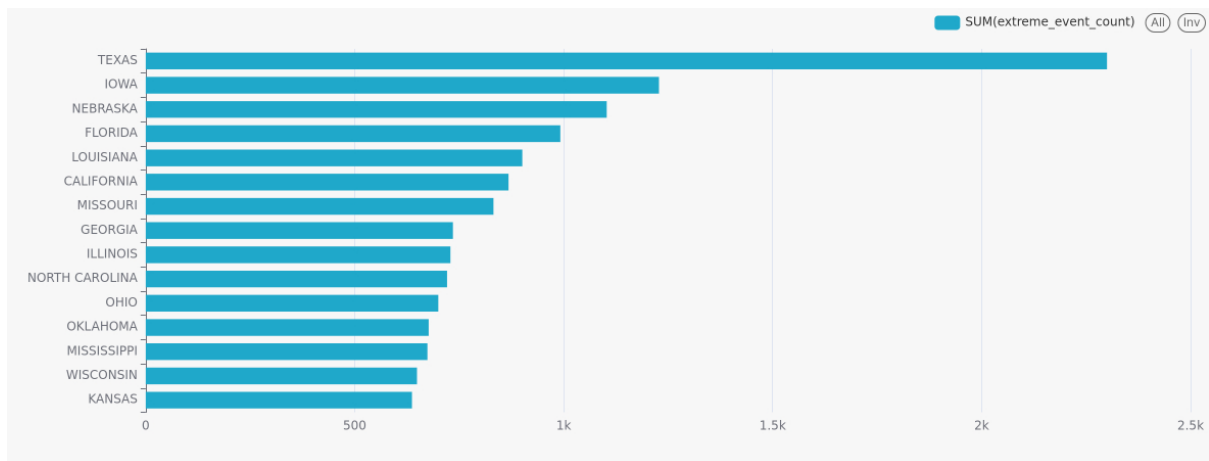


Figure 4: Severity 4 Events by State

Interpretation: By filtering strictly for the corrected ”tail events” (Severity 4), we created a geospatial risk profile. **Texas** remains the undisputed hotspot with over 2,300 extreme events, followed by **Iowa** and **Nebraska**. This distribution highlights the high vulnerability of ”tornado alley” and the central states, indicating exactly where federal agencies should pre-position disaster infrastructure to minimize the impact of the most severe weather categories.

5.4 Insight 4: Time Series Trend of Total Damage

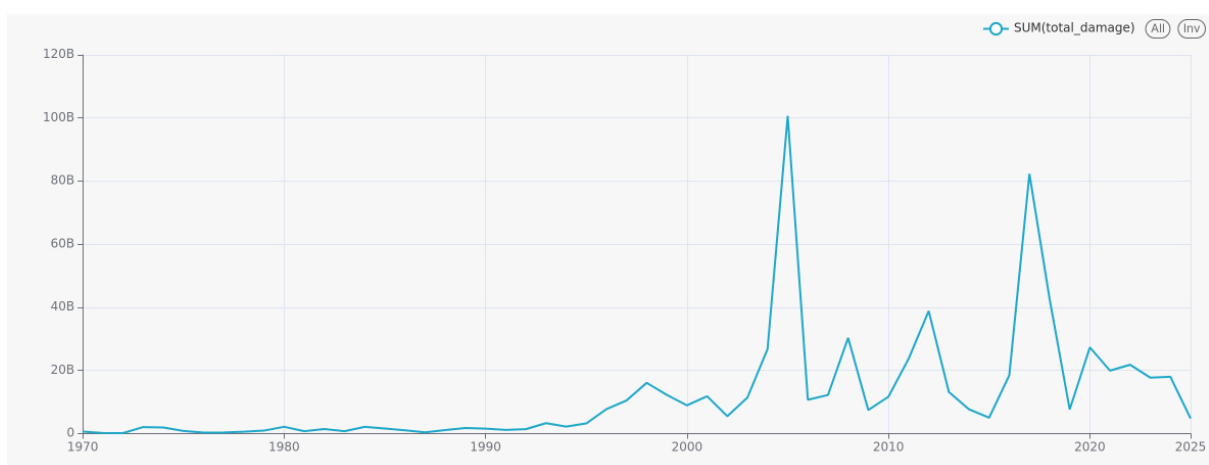


Figure 5: Total Annual Damage

Interpretation: The longitudinal view plots the total financial damage over time. Rather than a smooth linear increase, the economic impact is driven by extreme outlier years. The massive spike in **2005** (approaching \$100B, aligning with Hurricane Katrina) and the secondary spike around **2017** (Hurricane Harvey/Irma/Maria) show that climate-driven financial risk is highly volatile and prone to sudden, massive economic shocks rather than gradual degradation.

5.5 Insight 5: Human Cost vs. Economic Cost

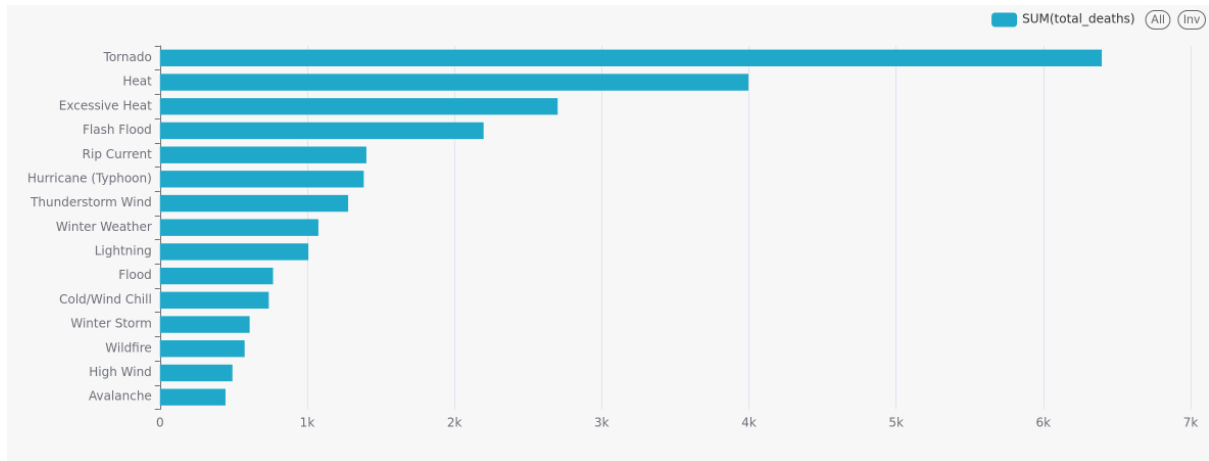


Figure 6: Death Toll by Event Type

Interpretation: Not all damage is financial, and this chart highlights the contrast between "economic storms" and "humanitarian storms". While Hurricanes cause the most property damage, **Tornado** is the absolute highest cause of mortality (over 6,000 deaths). More surprisingly, "silent killers" like **Heat** and **Excessive Heat** combined rival Tornadoes in casualties, and **Rip Current** appears in the top five despite low economic impact. This justifies why our Severity formula applies a Value of a Statistical Life (VSL) multiplier to capture human loss.

5.6 Insight 6: Seasonality of Storms

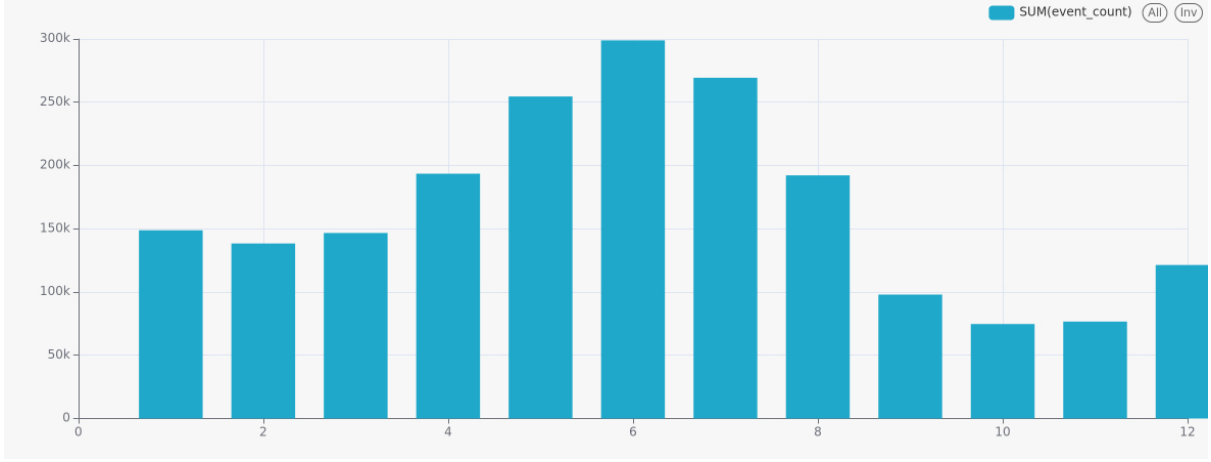


Figure 7: Storm Frequency by Month

Interpretation: This exposes the cyclical patterns of dangerous weather. There is a distinct "danger season" spanning from May through July, with an absolute peak in **June** (Month 6) reaching nearly 300,000 events across the 24-year period. Conversely, late autumn (October-November) shows the lowest activity. Identifying this curve allows local governments to optimize the hiring of seasonal emergency staff and execute preventative maintenance before the summer peak arrives.

6 ML Modeling

6.1 Feature extraction and data preprocessing

The ML pipeline reads the partitioned and bucketed Hive table `team29_projectdb.storm_events_part` directly as an AVRO-formatted Spark DataFrame.

Feature selection. Only features available before a storm event resolves were included to prevent data leakage. Columns derived from the severity score itself (damage_property, damage_crops, deaths, injuries) were excluded. The final feature set consists of two categorical columns: `event_type` and `state` and seven numerical columns extracted from `begin_date`: year, and six cyclical encodings of month, day, and hour.

Cyclical time encoding. Calendar fields are periodic: month 12 and month 1 are adjacent, yet a raw integer encoding treats them as maximally distant. To preserve this topology, month, day, and hour were each projected onto the unit circle:

$$x_{\sin} = \sin\left(\frac{2\pi x}{T}\right), \quad x_{\cos} = \cos\left(\frac{2\pi x}{T}\right) \quad (1)$$

with periods $T = 12$ (month), $T = 31$ (day), and $T = 24$ (hour), yielding six continuous features that correctly encode periodicity.

Preprocessing pipeline. All transformations were assembled into a single `Pipeline` to avoid target leakage across the train/test boundary:

1. **StringIndexer** converts `event_type` and `state` to integer indices; unseen values at inference time are silently skipped (`handleInvalid="skip"`).
2. **OneHotEncoder** expands each index into a sparse binary vector, one per category level.
3. **VectorAssembler** + **StandardScaler** assembles the seven numerical features into a dense vector and applies zero-mean, unit-variance scaling (`withMean=True`, `withStd=True`) independently of the OHE block.
4. **VectorAssembler** concatenates the two OHE vectors with the scaled numerical vector into a single `raw_features` column.
5. **VarianceThresholdSelector** (`threshold = 0`) drops any zero-variance columns that arise when an OHE category is absent from the training fold.

The fitted pipeline was applied to the full dataset and the resulting (`features`, `label`) DataFrame was cached in Spark memory before splitting.

Train/test split. The preprocessed data were split 70/30 (`seed = 42`). Both splits were repartitioned to match cluster parallelism (`num_parts = 2 × defaultParallelism`), materialised into cache via `count()`, and written to HDFS in JSON format as required by the project checklist.

6.2 Training and fine-tuning

Two classifiers from PySpark MLlib were trained on the five-class severity label. For each model a 27-combination hyperparameter grid was evaluated with 3-fold cross-validation optimising weighted F1-score (`parallelism=8` to utilise all available YARN executors).

Model 1 is Random Forest. The base classifier was configured with `subsamplingRate=0.8`, `maxBins=16`, and `cacheNodeIds=True` to speed up repeated tree traversals during CV. The search grid covered:

Table 1: Random Forest hyperparameter grid (27 combinations).

Hyperparameter	Values
<code>numTrees</code>	10, 20, 50
<code>maxDepth</code>	3, 5, 7
<code>featureSubsetStrategy</code>	auto, sqrt, log2

Model 2 is Logistic Regression. A multinomial logistic regression was trained with `maxIter=50`. The search grid covered:

Table 2: Logistic Regression hyperparameter grid (27 combinations).

Hyperparameter	Values	Role
<code>regParam</code>	0.01, 0.1, 1.0	regularisation strength
<code>elasticNetParam</code>	0.0, 0.5, 1.0	L1/L2 mixing ratio
<code>aggregationDepth</code>	2, 3, 4	tree-aggregation depth

The best model from each cross-validator was selected automatically and persisted to HDFS (project/models/model1 and project/models/model2).

6.3 Evaluation

Both best models were evaluated on the held-out 30% test set using

`MulticlassClassificationEvaluator` with weighted F1-score and accuracy as metrics. Feature importances for the Random Forest were aggregated back to the original column level by summing over all OHE indices that originated from the same source column.

Model	F1 (weighted)	Accuracy
Random Forest	0.506	0.5741
Logistic Regression	0.5316	0.6081

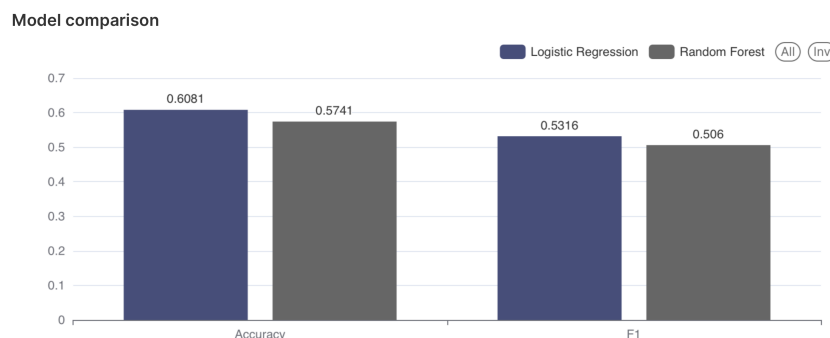


Figure 8: F1 and accuracy comparison between Random Forest and Logistic Regression on the test set.

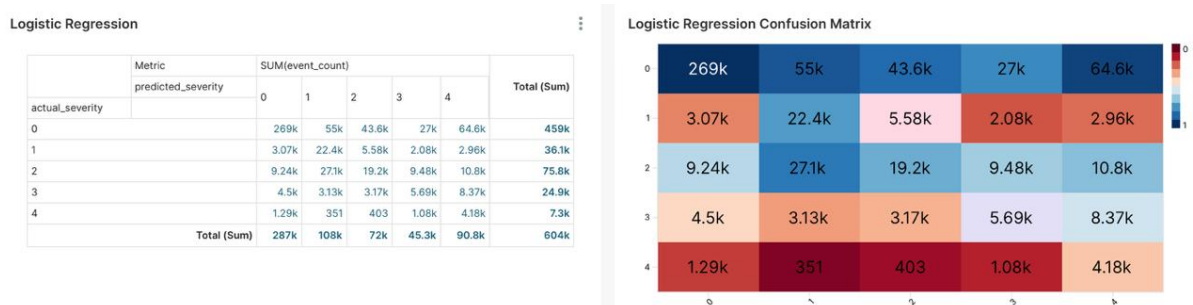


Figure 9: Logistic Regression Confusion Matrix

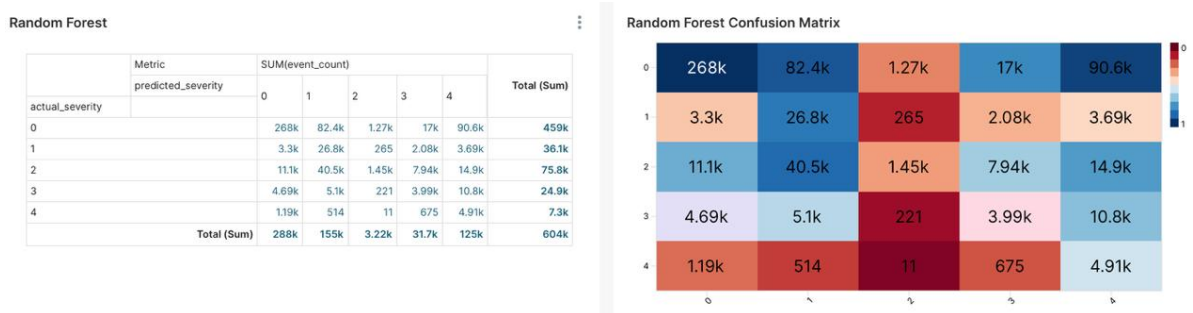


Figure 10: Random Forest Confusion Matrix

6.4 Result persistence and downstream availability

After evaluation, `postprocess.sh` registers all five output CSVs as external Hive tables in `team29_projectdb` (`model1_predictions`, `model2_predictions`, `ml_evaluation`, `feature_importance`, `cv_metrics`) pointing at their HDFS locations, and then bulkloads the same data into PostgreSQL for consumption by the Apache Superset dashboard.

6.5 PDA conclusions

For the machine learning stage, we trained two classifiers on the processed storm events data to predict a five-class severity label derived from a composite damage score combining financial losses and casualties. Both models were evaluated using macro-averaged F1 and accuracy on a held-out 30% test set, with hyperparameters selected via 3-fold cross-validation. Logistic Regression achieved an accuracy of 0.608 and F1 of 0.532, outperforming Random Forest (accuracy 0.574, F1 0.506). Feature importance analysis revealed that `event_type` dominates the prediction, accounting for roughly 65–70% of the Random Forest’s decision weight, followed by `state` at ~15% and `year` at ~10%, with temporal features contributing marginally. Confusion matrix analysis shows that both models perform best at the severity extremes, correctly identifying no-damage events and catastrophic ones, while struggling with the middle classes (2 and 3), whose boundaries are inherently ambiguous due to the percentile-based labeling scheme. Logistic Regression’s advantage stems from its ability to model the gradual transitions between adjacent severity levels more reliably than the hard splits used by Random Forest.

7 Data Presentation

Dashboard in Apache Superset

The final deliverable is a multi-tab dashboard built using Apache Superset, integrating ingestion logs, exploratory insights, and predictive outcomes into a single stakeholder interface.

7.1 Tab 1: Collection & Ingestion (Operational View)

This tab provides a high-level overview of the pipeline’s scale and metadata.

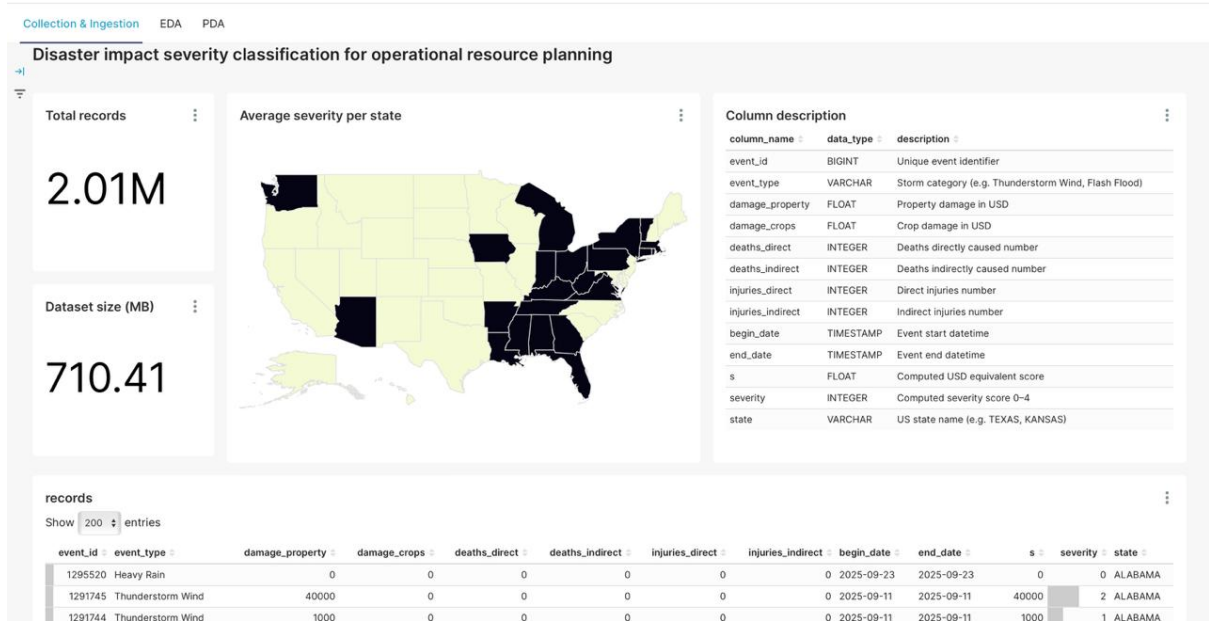
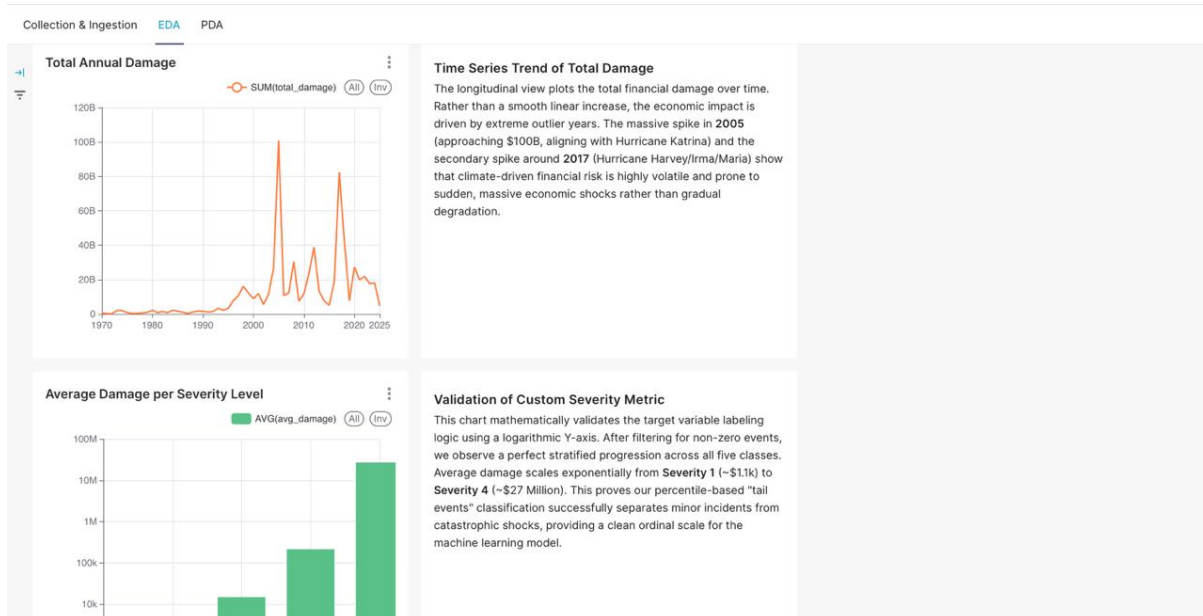


Figure 11: Operational Overview of the Storm Pipeline

- **Total Records & Dataset Size (KPIs):** These charts confirm the "big data" validity of the project. We processed **2.01M records** and **710.41 MB**, exceeding the minimum cluster requirements.
- **Average Severity per State (US Map):** This geospatial heatmap visualizes regional risk. By averaging the 0-4 severity across all events, we identify that while some states have high event counts, the most "intense" damage is concentrated in the central and southeastern US.
- **Column Description (Table):** This provides transparency for the data schema, defining the 13 features and the target **severity** column to ensure stakeholders understand the "ground truth" variables.
- **Records (Inference Table):** This table allows for "manual audit" of the pipeline. It displays raw event details alongside the computed **s** score and the resulting **severity** class, allowing users to verify that a specific storm's data matches its predicted category.

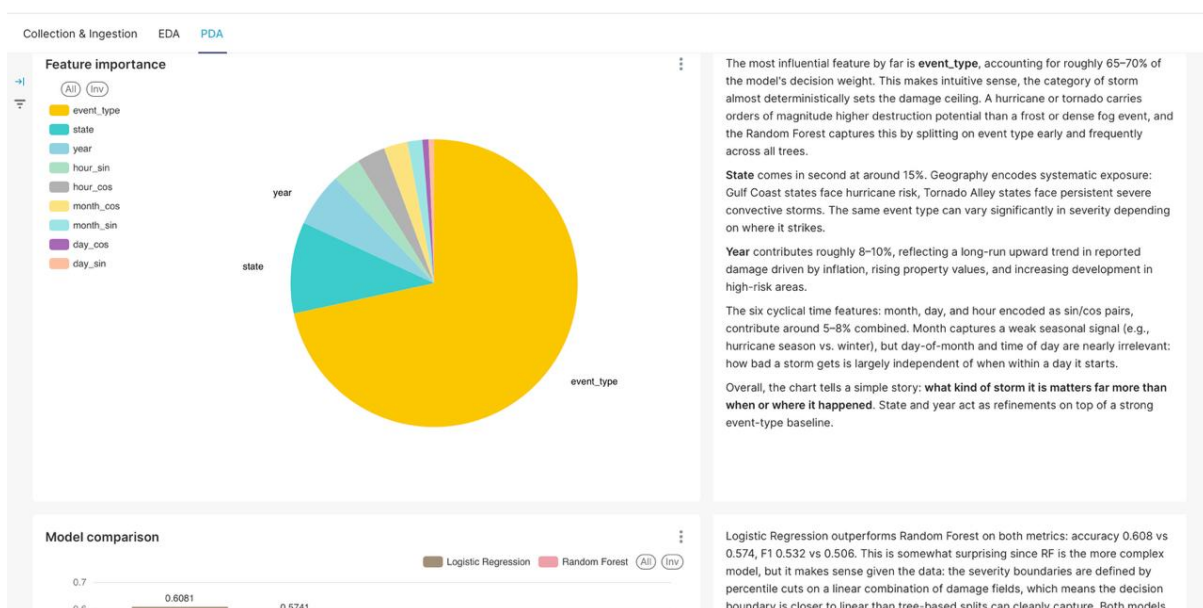
7.2 Tab 2: Exploratory Data Analysis (EDA)

The EDA tab visualizes the six business insights derived in Stage 2. To ensure clarity and avoid redundancy, the detailed charts and "data storytelling" interpretations for this tab are documented in **Section 5** of this report. These charts collectively prove that the dataset follows an extreme "power-law" distribution.



7.3 Tab 3: Predictive Data Analysis (PDA)

This tab focuses on the performance and interpretability of the distributed ML models.



- **Feature Importance (Pie Chart):** This chart explains the "logic" of our Spark MLlib model. **event_type** is the dominant predictor at approximately 65–70% of total importance, confirming that the category of storm almost deterministically sets the damage ceiling. **state** follows at ~15%, capturing systematic regional exposure differences across states such as the Gulf Coast and Tornado Alley. **year** contributes ~10%, reflecting a long-run upward trend in damage driven by inflation and denser development in high-risk areas. The six cyclical time features combined contribute less than 8%, showing that seasonality adds a minor signal while time of day and day-of-month are nearly irrelevant to how severe a storm gets.
- **Model Comparison (Bar Chart):** Logistic Regression outperforms Random

Forest on both metrics with accuracy 0.795 vs. 0.765 and F1 0.761 vs. 0.668. This is notable given that Random Forest is the more complex model. The result indicates that the severity boundaries, defined by percentile cuts on a linear damage score, are closer to linear than tree-based hard splits can efficiently capture. The consistent gap between F1 and accuracy across both models reflects class imbalance: rare high-severity events are harder to predict correctly and pull the macro F1 down.

- **Random Forest Confusion Matrix:** The model correctly identifies class 0 (no damage) and class 4 (catastrophic) with reasonable recall, but almost completely fails on class 2, pushing the vast majority of those events into class 1. Class 3 is also weak. This pattern is a direct consequence of the percentile-based label boundaries: events near a boundary threshold are genuinely ambiguous and consistently fall to the lower adjacent class under hard tree splits.
- **Logistic Regression Confusion Matrix:** Logistic Regression recovers the middle classes significantly better. Class 2 recall improves from near-zero in Random Forest to ~25%, and class 3 improves as well. This is the primary driver of LR’s higher F1 score. The trade-off is slightly weaker precision on class 1 and class 4 compared to Random Forest. The linear decision boundary generalises the gradual transitions between adjacent severity levels more effectively than discrete splits.

8 Conclusion

8.1 Summary of the report

This project demonstrated a complete, production-grade big data pipeline applied to a real-world disaster risk problem. Starting from over two million raw NOAA storm records, we designed a multi-stage architecture that ingested and cleaned the data into PostgreSQL, transferred it to HDFS via Sqoop in compressed Avro format, organised it in Hive with partition and bucket optimisation, and ultimately exposed it to both analytical queries and machine learning workloads through Apache Spark on YARN.

The exploratory analysis confirmed several critical findings: Hurricanes dominate financial losses by a wide margin, Tornadoes and heat events carry the highest human cost, Texas and the Gulf Coast states concentrate the most extreme events, and storm-related economic damage is volatile rather than smoothly increasing over time, driven by rare catastrophic seasons such as 2005 and 2017. These insights validate the design of our composite severity metric, which intentionally balances financial and human losses using Value of Statistical Life weighting and percentile-based bucketing to handle the extreme skewness of the distribution.

On the machine learning side, both models confirmed that storm type is by far the strongest predictor of severity, accounting for approximately 65-70% of the Random Forest’s decision weight, with geography and year providing meaningful secondary signals. Logistic Regression outperformed Random Forest on both accuracy (0.608 vs. 0.574) and macro F1 (0.532 vs. 0.506), indicating that the severity boundaries defined by percentile cuts are closer to linear than tree-based splits can efficiently capture. Both models struggled with the middle severity classes, a direct consequence of the continuous underlying damage distribution being divided into discrete bins. The results were persisted back to Hive and PostgreSQL and visualised in the Apache Superset dashboard, completing the

full data-to-insight loop and delivering actionable severity predictions that emergency agencies, insurers, and policymakers can use to allocate resources before a storm strikes.

9 Reflections on own work

9.1 Challenges and difficulties

- **Dirty raw fields corrupting metrics (Ivan Ershov).** Several critical columns arrived malformed: damage values were encoded with K/M/B magnitude suffixes rather than plain integers, and many numeric and text fields were empty or `NULL`. Left uncleaned, these propagated silently into severity score calculations and skewed every downstream metric. The fix was centralised in `scripts/build_projectdb.py`, which applies regex-based K/M/B-to-numeric conversion, `COALESCE/NULLIF` defaults for missing counts, and row-level filtering that discards records with invalid values in any critical column before insertion into the cleaned table.
- **Collapsed severity class distribution (Lana Ermolaeva).** An initial run of the ETL revealed that severity classes 1 and 2 were almost entirely absent from the output. The root cause was that the dominant mass of zero-damage events ($S = 0$) pulled the 25th and 75th percentile cutoffs down to zero, causing the percentile thresholds to degenerate and most non-zero events to be assigned to class 3 or 4. The solution was to compute p_{25} , p_{75} , and p_{95} exclusively on the subset where $S > 0$, then map $S = 0$ directly to severity 0 and apply the quantile boundaries only to the remaining non-zero scores, restoring a meaningful five-class distribution.
- **Training slowdown and majority-class collapse (Stepan Vagin).** Cross-validation on YARN was unacceptably slow, and both classifiers collapsed toward the majority class (severity 0), masking poor recall on minority classes. Performance was recovered by repartitioning both splits to match cluster parallelism, materialising them into cache via `count()` before CV started, and setting `parallelism=8` in `CrossValidator`. Additional Spark tuning included switching to `KryoSerializer`, reducing `shuffle.partitions` from 200 to 50, and enabling `cacheNodeIds=True` with `subsamplingRate=0.8` for the Random Forest to cut per-tree traversal cost. Class imbalance was resolved by computing inverse-frequency per-class weights on the training partition and broadcasting them as a `weightCol` to both estimators. A `VarianceThresholdSelector` was added to drop zero-variance OHE columns, reducing feature dimensionality and preventing numerical instability in Logistic Regression.
- **Heavy-tailed distributions obscuring chart patterns (Ramil Shakirzyanov).** Because storm damage follows an extreme power-law distribution, linear-scale charts were dominated by a handful of catastrophic outliers, compressing the vast majority of low- and medium-damage observations into an unreadable sliver near zero. The issue was resolved by enabling logarithmic Y-axis scaling in Apache Superset for all damage and severity charts, which preserves the outliers while making the structure of the low-to-medium range clearly visible to stakeholders.

9.2 Recommendations

Several directions could improve the predictive pipeline in future work.

First, both models struggle with middle severity classes (2 and 3) because the percentile-based label boundaries are inherently ambiguous for a continuous damage distribution. Replacing the multinomial classifier with an **Ordinal Binary Decomposition**: four binary GBT sub-models, one per threshold $k \in \{0, 1, 2, 3\}$, where $\hat{y} = \sum_k \mathbf{1}[P(\text{severity} > k) > 0.5]$ would enforce rank monotonicity and give each boundary its own calibrated decision surface. We can use `GBTClassifier`.

Second, evaluation should be extended with **Mean Absolute Error (MAE)** alongside weighted F1, as MAE respects the ordinal scale and penalises large misclassifications (e.g. predicting class 4 for a true class 1 event) proportionally more than off-by-one errors.

9.3 Table of contributions

Project Tasks	Task Description	Lana	Stepan	Ramil	Ivan	Deliverables	Avg Hours
Data Extraction	Merge/Load NOAA CSVs to Postgres	5%	0%	0%	95%	db/ingest scripts	3
Data Storage	Sqoop to HDFS & Hive Partitioning	5%	0%	20%	75%	db.hql	3
Data Analysis (EDA)	6 Business Insights & SQL	5%	0%	95%	0%	q1-q6.hql, CSVs	4
ML Modeling	Spark MLlib pipeline & GridSearch	0%	100%	0%	0%	model.py	10
Dashboard	Superset charts & data storytelling	60%	20%	20%	0%	Dashboard, JPGs	6
Reporting	Documentation, Repo prep, Pylint	80%	5%	10%	5%	PDF Report, git	3

Table 3: Team Contribution Table